

Тестирование

от unit до integration

Agenda

Зачем вообще нужны тесты

Пирамида тестирования

AAA и Given–When–Then

Unit-тесты на Kotlin: JUnit + MockK

Integration/UI tests в Android: Hilt + Compose + Navigation + Room

Что такое тестирование?

**Тесты - это код, который проверяет
корректность другого кода**

Капитан очевидность

Тестирование - это проверка того, что программа ведёт себя так, как мы ожидаем.

Капитан очевидность

Зачем оно нужно?

- Тестирование помогает ловить регресс
- Тестирование делает рефакторинг безопаснее
- Тестирование ускоряет обратную связь для разработчика
- Тестирование проверяет не только корректность, но и поведение и пригодность приложения к использованию

**тест - это автоматическая проверка
ожидаемого поведения программы**

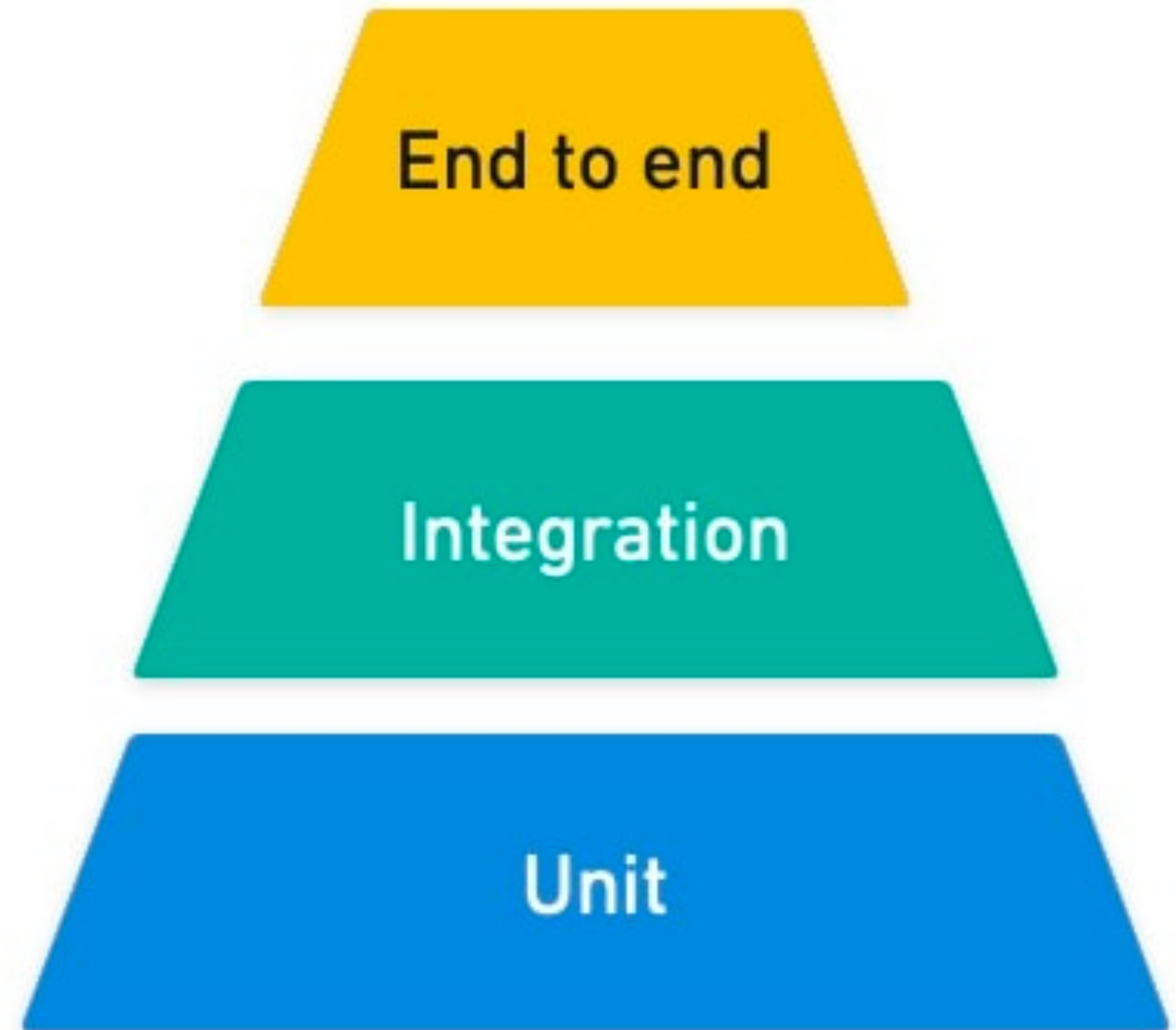
Какие бывают тесты в Android

- Local tests - запускаются на JVM на машине разработчика
- Instrumented tests - запускаются на устройстве или эмуляторе

Пирамида тестирования

- Внизу пирамиды - много маленьких unit-тестов
- Посередине - меньше integration-тестов
- Наверху - совсем немного end-to-end/UI-flow тестов

The Test Pyramid



Почему нельзя всё покрывать только UI-тестами

- UI-тесты ближе всего к реальному поведению пользователя
- Но они медленнее
- Они дороже в настройке и поддержке
- Их сложнее дебажить

Что такое unit test

- Unit test - это тест, который проверяет одну небольшую единицу поведения в изоляции от внешнего мира.
- unit test проверяет маленький кусок логики отдельно от остальной системы.

Что такое unit test

- Обычно unit-тест:
- быстрый
- маленький
- не требует устройства
- не ходит в сеть
- не работает с настоящей БД
- подменяет зависимости через fake/mock/stub

Что такое integration test

- Integration test - это тест, который проверяет, что несколько частей системы работают вместе.
- Примеры:
 - Compose screen + ViewModel + Hilt
 - NavHost + переход между экранами
 - Room DAO + база данных
 - Repository + fake API + Room

Что такое end-to-end test

- End-to-end test - это тест, который проверяет целый пользовательский сценарий от начала до конца.
- Примеры:
 - открыть приложение -> ввести данные -> нажать кнопку -> перейти на следующий экран
 - авторизоваться -> загрузить список -> открыть детали -> выйти назад

Сравнение: unit vs integration vs e2e

- **Unit**
 - тестирует один маленький кусок логики
 - быстрый
 - изолированный
 - дешёвый в поддержке
- **Integration**
 - тестирует взаимодействие нескольких частей
 - медленнее unit
 - ближе к реальному приложению
- **E2E**
 - тестирует целый пользовательский путь
 - самый дорогой
 - самый медленный
 - самый хрупкий

Теория и практика

Unit

Android

- В Android обычно:
 - `src/test` - unit-тесты
 - `src/androidTest` - integration / UI / instrumented tests

AAA и Given–When–Then

- **AAA = Arrange, Act, Assert**
- **GWT = Given, When, Then**
- По сути это два близких способа структурировать тест
- AAA удобнее для unit-тестов
- GWT удобнее для объяснения сценариев поведения и acceptance-thinking

Пример AAA / GWT

- **AAA**
 - Arrange: создать ViewModel с fake repository
 - Act: вызвать loadData()
 - Assert: проверить, что UI state содержит список
- **GWT**
 - Given ViewModel с поддельным репозиторием
 - When пользователь инициирует загрузку
 - Then экран показывает данные

test double

- Test double - это подставной объект вместо настоящей зависимости
- Основные виды:
 - fake - рабочая, но тестовая реализация, например in-memory база;
 - mock - объект с ожиданиями по взаимодействию;
 - stub - объект, который просто возвращает заранее заданные значения;
 - spy - обёртка над реальным объектом.

JUnit

- JUnit - это framework для автоматического тестирования Java/Kotlin-кода
- Позволяет писать и запускать тесты
- Даёт структуру тестов: тестовые методы, подготовку, очистку
- Даёт assertions для проверки результата
- Интегрируется с IDE, Gradle и CI
- В Android чаще всего используется JUnit 4

Основные аннотации JUnit

- `@Test` - тестовый метод
- `@Before` - выполняется перед каждым тестом
- `@After` - выполняется после каждого теста
- `@BeforeClass` - один раз перед всеми тестами класса
- `@Rule` - подключение rule-объектов JUnit/AndroidX Test

Пример теста

```
class UserViewModelTest {

    private lateinit var repository: UserRepository
    private lateinit var viewModel: UserViewModel

    @Before
    fun setUp() {
        repository = mockk()
        viewModel = UserViewModel(repository)
    }

    @Test
    fun loadUsers_success_updatesState() = runTest {
        every { repository.getUsers() } returns listOf(User("Alice"))

        viewModel.loadUsers()

        assertEquals(1, viewModel.uiState.value.users.size)
    }
}
```

Mockito vs Mockk

- Mockito можно использовать в Kotlin
- Но для Kotlin часто подключают ещё mockito-kotlin
- MockK позиционируется как mocking library for Kotlin
- У MockK есть удобные coroutine API: coEvery, coVerify

Integration

Android-специфика integration tests: что добавляется поверх JUnit

- `@RunWith(AndroidJUnit4::class)`
- `@HiltAndroidTest`
- `@Before fun init() { hiltRule.inject() }`

Hilt в тестах: основные аннотации и приёмы

- `@HiltAndroidTest` — включает Hilt-компоненты для теста
- `HiltAndroidRule` — управляет состоянием компонентов и инъекцией
- `hiltRule.inject()` — делает field injection в тест
- `@BindValue` — быстро подставить fake/mock в граф